

To set up the **Builder and Validator pairing**, you must define these as specialized roles within your multi-agent team, typically designated in a team task list or orchestration prompt 1, 2. The **Builder agent** handles core implementation, feature development, and file manipulation, while the **Validator agent** is responsible for writing tests, running validation scripts, and ensuring code quality through independent verification 2, 3. To enhance reliability, you can configure **self-validation hooks** (such as ruff or mypy) within the Builder agent to run automatically after each file edit, while the Validator acts as a higher-level gate to catch errors before a pull request is opened 4, 5.

The **six key functions** of the task system, optimized as 6-letter verbs for brevity and token reduction, are:

1. **PLAN** : Generates the task graph or Directed Acyclic Graph (DAG) for the work 6, 7.
2. **SRCH** : Retrieves relevant code symbols, file references, or line numbers 6, 7.
3. **PACH** : Applies AST-aware code patches or modifications to files 6, 7.
4. **TEST** : Executes test runners (like pytest or jest) and reports the status 6, 7.
5. **RISK** : Analyzes proposed changes to provide a risk score based on complexity and code churn 6, 7.
6. **PR** : Handles Git operations to open and manage Pull Requests 6, 7.

Nano Banana generates UI styles and images by interpreting natural language descriptions to create professional visual designs 8, 9. You can prompt the system to generate a specific **UI layout**, such as "three vintage themed website designs," and it will implement the chosen aesthetic using clean fonts and minimal layouts 8, 10. Furthermore, it allows for **image generation** directly within the IDE; for example, you can ask it to "create a neon futuristic world city" to fill empty placeholders or generate unique **app icons** for deployment 11-13. It can also perform **AI-powered image editing**, such as removing backgrounds or changing specific element colors in seconds 14.